SHIELD TECHNOLOGIES

Fruit Shop POS & Inventory Product

TECHNICAL ARCHITECTURE DOCUMENT

Vue.js + Laravel + MySQL - Microservices Architecture

Derived from the Software Requirements Specification (v2.0) and the REST API Documentation (v1.0)

Stack: Vue.js | Laravel | MySQL

Intern Training Project - Day 2 | Development Phase | 16 October 2026

1. Introduction and Purpose

This Technical Architecture Document defines the system design for Shield Technologies' subscription-based Fruit Shop POS & Inventory product. It translates the SRS (functional requirements FR-01 to FR-25, non-functional requirements NFR-01 to NFR-08, acceptance tests AT-01 to AT-08) and the REST API contract (base path `/api/v1`) into a concrete technology and component design using a **Vue.js frontend, Laravel microservices backend and MySQL database**.

1.1 Business Objective

Give fruit retailers a reliable web POS that sells by weight or count, tracks every kilogram through batches, ripeness and locations, controls spoilage, and keeps stock and money consistent under concurrent use, while each shop's data stays isolated from every other subscriber.

1.2 In-Scope Capabilities

- Tenant onboarding, staff accounts, four roles (SA, OM, SC, CA) and subscription administration (plans, verified payments, suspend, cancel)
- Products (KG / EACH), categories, locations, suppliers, price history and reorder settings
- Purchase orders, goods receipts with gross/tare/net weights, accepted vs rejected quantity, and batch creation
- Batch and stock-position control: FEFO allocation, ripeness and quality state, quarantine, transfers, repacking, opening stock
- Wastage, supplier returns and stock counts with owner approval and position locking
- POS checkout (cash and externally confirmed card), receipts (JSON/HTML/PDF, A4 and 80 mm), customer returns and refunds
- Optional customers, cashier shifts and cash movements
- Dashboards, reports (sales, stock, ageing, wastage, margins, purchases, count variances), CSV exports and audit history

1.3 Explicit Constraints

- Frontend: Vue.js. Backend: Laravel services. Database: MySQL
- Architecture style: microservices with database per service, matching SRS section 13 (four services)
- Out of release scope: automatic scale/scanner/payment-gateway/accounting integration, multi-branch, offline mode, credit sales, loyalty, mixed-fruit bundles
- No card numbers, security codes or payment credentials are ever submitted or stored; the API only records externally confirmed payments
- Internet connectivity is required; browser support is current and previous Chrome and Edge
- One shop per tenant; all business records and operations are tenant scoped

2. System Architecture Diagram

The diagram shows the complete architecture: a Vue SPA calling a single Laravel API Gateway, four independently deployable Laravel services, and a MySQL server hosting one database per service.

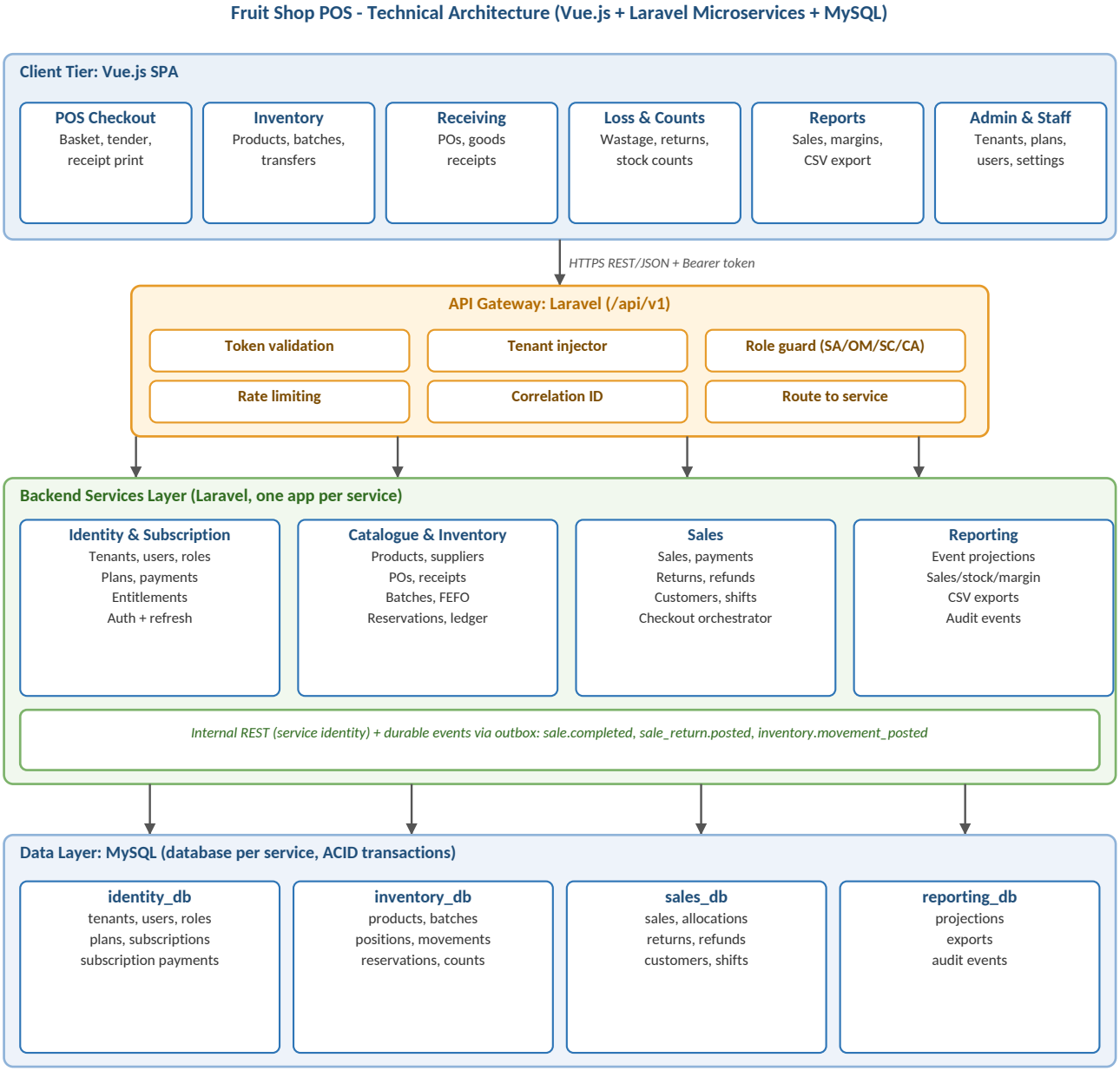


Figure 1 - Fruit Shop POS Technical Architecture (Vue.js + Laravel Microservices + MySQL)

3. High-Level Microservices Architecture

The system follows the four service boundaries defined in SRS section 13.1. Each service owns its data, exposes a REST API, and can be developed, deployed and scaled independently. A lightweight Laravel gateway routes browser traffic and applies cross-cutting concerns. Catalogue and stock are deliberately kept in one service so receiving, counting, reservations and ledger writes stay atomic inside one MySQL transaction.

3.1 Logical Layers and Services

Layer / Service	Technology	Primary Responsibility
Frontend (SPA)	Vue.js	Cashier POS, inventory and owner screens, reports, Shield admin console
API Gateway	Laravel	Routing, token validation, tenant and role enforcement, rate limiting, request logging
Identity & Subscription Service	Laravel + MySQL	Tenants, shop settings, users/roles, auth sessions, plans, subscriptions, subscription payments, entitlements
Catalogue & Inventory Service	Laravel + MySQL	Products, categories, suppliers, locations, POs, receipts, batches, positions, reservations, movements, counts, wastage, supplier returns
Sales Service	Laravel + MySQL	Sales, lines, allocations, retail payments, receipts, returns/refunds, customers, shifts, cash movements
Reporting Service	Laravel + MySQL	Read-only projections from events, dashboard, reports, CSV exports, audit event store
Background jobs	Laravel	Reservation expiry, checkout and return reconcilers, event publication, alert refresh

3.2 Core Design Principles

- Database per service: no service reads or writes another service's database; cross-service references are opaque UUIDs
- Server-authoritative prices, discounts, taxes, totals, availability and state transitions; client-supplied totals are never trusted
- Tenant derived from the authenticated identity, never from a client-supplied tenant_id; every query is filtered by tenant
- Inventory owns stock consistency; Sales orchestrates checkout; completion only after Inventory confirms the reservation commit
- Idempotency-Key on every stock-changing and financial POST; ETag / If-Match on mutable documents
- Immutable ledger: stock movements, posted documents and audit events are never edited, only corrected by linked reversals
- Decimal-safe arithmetic: quantities, money and costs are never handled as binary floating point
- Unknown outcomes stay PENDING and are reconciled with the same request identifier; no silent completion or silent retry with a new key
- Services communicate by synchronous REST for request/response and by durable at-least-once events for side effects

4. Frontend Architecture (Vue.js)

The frontend is a single Vue.js SPA. It communicates only with the API Gateway and never calls an individual service. The tablet checkout layout works from 768 px and owner views from 360 px (NFR-06).

4.1 Route Structure

Route	Role	Screens
/login, /forgot-password, /reset-password	Public	Sign in and password reset
/pos/*	CA, OM	Shift open/close, basket, checkout, receipt reprint, sale search, customer lookup
/products, /categories, /locations	OM; SC/CA read	Product list and form, price history, categories, locations
/suppliers, /purchase-orders	OM; SC read	Supplier list, PO create/approve/close/cancel
/receiving/*	OM, SC	Goods receipt draft (gross/tare/net), post, reverse
/inventory/*	OM, SC (CA availability)	Availability, batches, positions, movements, alerts, transfers, assessments, opening stock
/loss/*, /counts/*	OM, SC	Wastage, supplier returns, stock counts and approvals
/returns/*	OM	Customer return/refund draft and post, quarantine release
/reports/*, /exports, /audit	OM	Reports, CSV exports, audit history
/settings, /staff, /subscription	OM	Shop settings, staff accounts, plan and renewal date
/admin/*	SA	Tenants, plans, subscription payments, suspend/cancel

4.2 Key Frontend Responsibilities

- Show unit labels (kg / item) beside every quantity and price; enforce 3-decimal KG and whole-number EACH inputs before submit
- Display server-returned prices and totals; client calculations are indicative only
- Hide costs, margins and supplier costs from cashiers; role-based menus and route guards mirror (but never replace) server permissions
- Hold the checkout idempotency key in state; on timeout or 503, poll `GET /sales/{id}` instead of creating a new checkout
- Handle 401 by refreshing the session; handle 403 SUBSCRIPTION_INACTIVE by switching to read/export-only mode
- Handle 412 VERSION_CONFLICT by reloading the document and asking the user to reconcile
- Render receipts from JSON for 80 mm and A4 browser printing; reprint never creates a new sale
- Manual entry of scale readings, external card confirmation and tare/gross weights, as required by the SRS

5. Backend Microservices (Laravel)

Each microservice is an independent Laravel application with its own MySQL database. Services expose versioned REST endpoints; internal routes accept only authenticated service identities.

5.1 Service Responsibilities

Identity & Subscription Service (FR-01, FR-02, FR-03, FR-25 settings)

- Login, refresh, logout, forgot/reset password, `/auth/me`, staff management (owners cannot grant SA)
- Tenants, plans, subscription payments (idempotent, no gaps or duplicate entitlement), suspend, cancel, shop settings
- Internal entitlement check consumed by all other services

Catalogue & Inventory Service (FR-04 to FR-14)

- Products, categories, locations, suppliers, price history, reorder suggestions
- Purchase orders and goods receipts; posting atomically creates batches, positions and movements
- Availability, FEFO reservations (create, commit, release, expiry), transfers, repacking, quality assessments, quarantine release
- Wastage, supplier returns, stock counts with position locks; ledger and alerts

Sales Service (FR-15 to FR-21, FR-24)

- Draft basket, server-side pricing/discount/tax, checkout orchestration, payment record, receipts
- Customer returns and refunds, cumulative return limits, customers, cashier shifts and cash movements

Reporting Service (FR-22, FR-23, FR-25)

- Consumes sale.completed, sale_return.posted, inventory.movement_posted; deduplicates by event_id; exposes projection timestamp (data_as_of)
- Dashboard, reports, CSV export jobs with short-lived protected download references, audit events (append-only)

5.2 API Gateway Routing

Public path prefix (/api/v1)	Routed to	Access note
/auth/*, /users, /subscription, /settings, /admin/*	Identity & Subscription	/admin/* restricted to SA
/products, /categories, /locations, /suppliers, /purchase-orders, /goods-receipts, /inventory/*, /batches, /wastage-records, /supplier-returns, /stock-counts	Catalogue & Inventory	Role matrix per endpoint; cost fields stripped for CA
/sales, /sale-returns, /customers, /shifts	Sales	OM, CA (own shift for CA)
/dashboard, /reports/*, /exports, /audit-events	Reporting	Mostly OM; dashboard role filtered
/internal/v1/*	Not exposed	Service-to-service only, blocked at the gateway

5.3 Critical Cross-Service Behaviours

- Stock is changed only by the Catalogue & Inventory service; Sales never writes inventory tables
- Checkout and return flows use internal reservation / sale-return-receipt endpoints with the same request ID for idempotency
- Reservation expiry (default 5 minutes) and commit compete atomically; only one wins

- Identity or Inventory failure blocks new checkout when authorization or availability cannot be established; Reporting outage never blocks sales
- Expired or suspended subscriptions block new writes (403 SUBSCRIPTION_INACTIVE) while owner read/export remains; recovery workers may finish previously authorized transactions

5.4 Layers inside Each Laravel Service

Layer	Laravel element	Responsibility
HTTP	Routes, Controllers, Requests	Validation (422), response envelope data/meta/error, ETag headers
Security	Middleware	Token + service identity, tenant scope, role permission, subscription check, idempotency, If-Match
Domain	Service classes	Business rules: FEFO, rounding, state machines, return limits
Persistence	Models	Tenant scope, decimal values, transactions (InnoDB)
Async	Jobs, Outbox publisher, Event consumers	Expiry, reconcilers, event publication and idempotent handling

6. Database Structure (MySQL - Database per Service)

Each service has its own MySQL database (schema). Cross-service references use UUIDs only, never foreign keys across databases. InnoDB transactions are used for receiving, reservations, commits, counts and returns so each completes atomically.

6.1 Service Data Ownership

Service / Database	Owned Tables
Identity & Subscription (identity_db)	tenants, users, roles, refresh_sessions, plans, subscriptions, subscription_payments, settings
Catalogue & Inventory (inventory_db)	products, price_history, categories, suppliers, locations, purchase_orders, goods_receipts, batches, stock_positions, stock_movements, reservations, quality_assessments, wastage_records, supplier_returns, stock_counts, idempotency_keys, outbox
Sales (sales_db)	sales, sale_lines, allocations, retail_payments, sale_returns, return_lines, refund_records, customers, shifts, cash_movements, idempotency_keys, outbox
Reporting (reporting_db)	sales_projection, stock_projection, margin_projection, wastage_projection, export_jobs, audit_events, processed_events

6.2 Indexing and Integrity Strategy

- Unique: products (tenant_id, sku); suppliers (tenant_id, supplier_code); sales (tenant_id, sale_number); sales (tenant_id, request_id)
- Unique: stock_positions (tenant_id, batch_id, location_id); idempotency_keys (tenant_id, operation, caller, key)
- Reservations carry an expiry time (default five minutes); idempotency keys are retained at least seven days
- Stock changes and reservations are concurrency safe and idempotent
- Version column on mutable records for ETag / If-Match; posted records are immutable
- Soft deactivation of products, suppliers and locations; referenced rows are never hard-deleted
- Snapshots: price, tax, discount and batch cost are copied onto sale lines and allocations so history survives later changes

6.3 Data Type Rules

Data	MySQL type / rule	Notes
Money (2 dp)	DECIMAL(12,2), half-up rounding	Decimal strings in the API
Cost rates	DECIMAL(18,6)	Unit cost = accepted line cost / accepted quantity
KG quantity	DECIMAL(14,3)	Never floats; EACH quantities must be whole numbers
Dates / timestamps	ISO 8601 UTC; business dates in tenant timezone	YYYY-MM-DD for calendar dates
Identifiers	UUID (CHAR(36))	tenant_id column on every tenant-scoped table

7. Project Folder Structure

The repository is a multi-service monorepo. Each Laravel service has the standard Laravel layout and communicates with the others only over the network. The same structure is created in the GitHub repository on the team leader's `dev` branch.

```
fruit-shop-pos/
|-- frontend/                Vue.js SPA
|   |-- src/
|   |   |-- api/             calls the API Gateway only
|   |   |-- components/
|   |   |-- views/           pos, inventory, receiving, loss, returns, reports, settings, admin
|   |   |-- router/
|   |   |-- stores/
|   |   `-- utils/
|   `-- package.json
|-- gateway/                 Laravel API Gateway
|   |-- app/Http/Middleware/
|   |-- routes/api.php
|-- services/
|   |-- identity-service/    Laravel (tenants, users, subscriptions)
|   |-- inventory-service/   Laravel (catalogue and inventory)
|   |-- sales-service/       Laravel (sales, returns, customers, shifts)
|   `-- reporting-service/   Laravel (projections, reports, exports, audit)
|-- docs/                    SRS, API documentation, this document
`-- README.md
```

8. Component Communication

8.1 Browser to API Gateway

- HTTPS REST JSON only; `Authorization: Bearer <access_token>` (access token expires in 900 seconds)
- Refresh session via Secure, HttpOnly cookie with SameSite policy and CSRF protection; logout revokes the session
- Idempotency-Key on stock-changing and financial POSTs; If-Match / ETag on versioned documents
- Responses follow the documented envelope: data / meta (page, limit, total) / error (code, message, details, request_id)
- Status codes: 200, 201, 202, 204, 400, 401, 403, 404, 409, 412, 422, 428, 429, 503

8.2 Inter-Service Communication

- Internal endpoints are available only to authenticated services; tenant context is asserted by a trusted service identity and checked against all referenced records
- Internal endpoints: entitlements, reservations (create, by-request, commit, release), sale-return-receipts
- Asynchronous durable events: sale.completed, sale_return.posted, inventory.movement_posted
- Each event carries event_id, event_type, schema_version, tenant_id, occurred_at, correlation_id, data
- Producers use an outbox or equivalent reliable publication
- Delivery is at least once; consumers deduplicate by event_id

8.3 Checkout Flow

- 1. Cashier submits `POST /sales/{id}/checkout` with Idempotency-Key; Sales checks entitlement and creates/loads the PENDING sale with its request ID
- 2. Sales calls Inventory `POST /internal/v1/reservations`; Inventory validates stock, locks and quality and reserves FEFO quantities atomically (default 5 minutes)
- 3. Sales records the payment (cash tender covers total, or CARD with externally_confirmed=true)
- 4. Sales calls reservation commit; Inventory deducts stock once and posts sale movements in one transaction
- 5. Sales marks the sale COMPLETED, stores allocations and cost snapshots, returns 200 with totals and change, and writes the sale.completed outbox event
- 6. On timeout the API returns 202 with status_url; the client reads the existing sale; the reconciler uses the same request ID to finish or release

8.4 Goods Receipt Flow

- Clerk drafts a receipt (gross, tare, delivered, accepted, rejected, cost, location, batch data); no stock effect while draft
- Post validates gross minus tare, accepted plus rejected equals delivered, and order limits (excess or direct receipts need owner authorization)
- One MySQL transaction creates batches, positions and movements for accepted stock only, and updates purchase order totals; retry adds nothing

8.5 Customer Return Flow

- Owner creates a return draft; Sales checks cumulative quantity and value limits using original price/discount/tax snapshots
- Post (Idempotency-Key + If-Match) calls Inventory sale-return-receipts to quarantine returned goods, then records the externally confirmed refund
- Returns stay PENDING until stock action and refund record are confirmed; a reconciler recovers partial failures; repeats never duplicate refund or stock

9. Security, Concurrency and Cross-Cutting Concerns

9.1 Security

- TLS 1.2 or later everywhere; encrypted storage and backups
- Salted adaptive password hashes; login rate limits; reset requests return a generic response
- Expiring access tokens plus a Secure, HttpOnly refresh cookie; disabled users lose protected access
- Tenant isolation: tenant derived from token; tenant scope on every query; 404 RESOURCE_NOT_FOUND for cross-tenant IDs (no existence disclosure)
- Role matrix enforced by gateway and again in each service; cost, margin and supplier cost fields removed from CA responses
- No password, token or payment credential is logged; Shield support access requires explicit role and is audited
- Internal endpoints are not browser APIs and are not publicly reachable

9.2 Concurrency and Integrity

- Stock changes and reservations are atomic, concurrency safe and idempotent
- Overselling is prevented under concurrency: two cashiers competing for the last 1.000 kg, only one reservation succeeds (AT-03)
- Stock count locks scoped positions; reserved positions cannot enter a count; reserved stock cannot be wasted, returned or moved
- Idempotency mappings scoped by tenant, operation and caller; same key with a different payload returns 409 IDEMPOTENCY_CONFLICT
- Optimistic concurrency: stale If-Match returns 412, missing precondition 428
- Rounding: line amount, discount and tax rounded half-up to two decimals, totals are the sum of rounded lines (worked example: 1.250 kg x 650.00, 10% discount gives 731.25, change 268.75)

9.3 Observability

- Correlation / request ID created at the gateway and propagated to every service and event
- Central logs with tenant and correlation IDs, without passwords, tokens or card credentials
- Metrics and alerts: checkout failures, stock discrepancies, expiry jobs, projection lag, unresolved transactions (alert within five minutes)
- Health alerts detect failures within five minutes

9.4 Error and Retry Handling

Code	HTTP	Use
VALIDATION_FAILED	422	Field-level details
INSUFFICIENT_STOCK	409	Requested and available quantity
STOCK_COUNT_LOCKED	409	Scope reference
INVALID_DOCUMENT_STATE / RETURN_LIMIT_EXCEEDED	409	Current state / remaining returnable quantity
IDEMPOTENCY_CONFLICT / VERSION_CONFLICT	409 / 412	Key reuse with different payload / stale If-Match
SUBSCRIPTION_INACTIVE / FORBIDDEN	403	Owner may still view and export / role not permitted
RESOURCE_NOT_FOUND	404	No cross-tenant disclosure

10. Deployment Considerations

- Each service is independently deployable; Shield Technologies owns hosting, backup, subscription administration and operational monitoring
- The browser uses an HTTPS API gateway as the single public entry point
- MySQL with InnoDB is used so transactions are available
- Daily encrypted backups retained 30 days (proposed RPO 24 hours, RTO four hours); restore verified before launch and quarterly
- Targets: 99.5% monthly availability; product/stock searches within 2 seconds; checkout within 3 seconds; 100 active tenants and 50 concurrent users platform-wide
- Versioned APIs, service documentation, automated critical-flow tests and deployment rollback
- Browser printing for A4 and 80 mm receipts; current and previous Chrome and Edge
- Deployment host, limits, retention and allowed state changes are finalized during implementation review

10.1 Technology Summary

Concern	Choice
Frontend	Vue.js
Backend	Laravel services plus a Laravel API gateway
Database	MySQL, one database per service
API	JSON REST, base path /api/v1
Receipts	JSON, HTML and PDF; paper A4 or 80 mm

11. Requirements Traceability

The architecture supports all functional requirements of SRS v2.0 and the endpoints of API v1.0.

SRS requirements	Owning service	Main API area	Acceptance
FR-01 to FR-03, FR-25 (settings)	Identity & Subscription	/auth, /users, /admin, /subscription, /settings	AT-01
FR-04 to FR-07	Catalogue & Inventory	/products, /categories, /locations, /suppliers	AT-01, AT-02
FR-08, FR-09	Catalogue & Inventory	/purchase-orders, /goods-receipts, /inventory/opening-balances	AT-02
FR-10, FR-11, FR-14	Catalogue & Inventory	/inventory, /batches, /internal/v1/reservations	AT-03
FR-12, FR-13	Catalogue & Inventory	/wastage-records, /supplier-returns, /stock-counts	AT-04
FR-15 to FR-18	Sales (+ Inventory)	/sales, receipt endpoint	AT-05
FR-19, FR-20	Sales (+ Inventory)	/sales/{id}/returns, /sale-returns	AT-06
FR-21, FR-24	Sales	/customers, /shifts	AT-07
FR-22, FR-23, FR-25 (audit)	Reporting	/reports, /exports, /dashboard, /audit-events	AT-07
NFR-01 to NFR-08	All services, gateway, deployment	Sections 8 to 10	AT-08

Acceptance scenarios for tenant isolation, concurrency, idempotency, FEFO, rounding and reconciliation remain fully applicable remain fully applicable.

12. GitHub Repository and Team Workflow

Branch	Owner	Purpose
main	Team	Stable, reviewed releases only
dev	Team Leader	Integration branch; initial folder structure from Section 7
member-1-name, member-2-name, member-3-name	Each member	Individual development; merged into dev through reviewed pull requests

- The Team Leader creates the repository and the dev branch, and sets up the initial folder structure with 3 members
- The other 2 members and the `yohanmri` account are added as collaborators
- Each of the 3 team members creates a separate branch using their own name; completed work is merged into dev after review

12.1 Decisions to Confirm

- Subscription prices/intervals, trial/grace policy (none assumed), stock clerk posting permissions
- Scale-entry precision, default saleable ripeness, alert windows and retention policy
- Supported printer, production tenant/load targets and purchase cost treatment

12.2 Document Control

Field	Value
Owner	Shield Technologies
Product	Fruit Shop POS & Inventory Product
Document	Technical Architecture Document - Vue.js + Laravel + MySQL Microservices
Version / Date	1.0 / 16 October 2026
Based on	SRS v2.0 (14 Oct 2026), REST API v1.0 (15 Oct 2026)
Stack Constraint	Vue.js + Laravel + MySQL
Architecture Style	Microservices (database per service)
Status	Draft for team review